PROJECT REQUIREMENT DOCUMENT · #63

Anurag D

DISCOVERERS G3

Namma-Hasiru

Sustainability & Reforestation Tracker — Android App

Kotlin | Firebase Firestore | FusedLoaction | WorkManager | Glide | Android Studio

PROJECT	TIMELINE	PLATFORM
#63	3 Weeks	Android

1 What Is This App? (Simple Overview)

Namma-Hasiru is an Android app that acts as a digital guardian for reforestation efforts. Think of it as a **"Health Record"** for every tree planted by **volunteers**, turning a one-time planting event into a **long-term survival** mission.

Volunteers use the app to log new saplings or seed balls with a single tap **(photo + GPS)**. Communities can then track these plants over months, ensuring they don't just get **"thrown and forgotten"**, but actually grow into a forest.

2 What Problem Does It Solve?

Most plantation drives focus on the number of seeds *thrown*, not the number of trees *grown*. Without tracking, there is zero data on survival rates, leading to wasted resources and failed ecological goals.

Namma-Hasiru fixes this—it creates accountability through geo-tagging and automated reminders, allowing volunteers to monitor growth and providing data on which species actually thrive in specific soils.

WITHOUT THIS APP

- ✓ "Plant and forget" mentality.
- ✓ No data on whether seeds sprouted or died.
- ✓ Zero accountability for community drives.
- ✓ Unknown survival rates for different regions.
- ✓ Resources wasted on unsuitable tree species.

WITH THIS APP

- ✓ Every sapling is geo-tagged and "owned."
- ✓ 90-day reminders to check and update status.
- ✓ Visual proof of growth through "Growth Photos."
- ✓ Community "Survival Scores" (e.g., 70% success).
- ✓ Data-driven guides on the best trees for local soil.

3 Scope — What's In and What's Out

IN SCOPE — We Will Build This

- → Planting Log Form: Species name + Photo capture.
- → GPS Integration: Automatic geo-tagging using FusedLocation.
- → Tree Map: A community view showing all saplings on a map.
- → 90-Day Alarm: Automated status update alerts via WorkManager.
- → Survival Dashboard: Survival percentage for the village/region.
- → Local Database: Room DB for offline history and plant tracking.
- → Species Guide: Basic list of native trees based on success data.
- → Earthy UI: A green, nature-inspired Material Design interface.

OUT OF SCOPE — Won't Build

- → AI Plant Disease Detection: Identifying pests via camera.
- → Satellite Imagery: Real-time forest monitoring from space.
- → In-App Marketplace: Selling saplings or tools.
- → Social Network: Full profiles, comments, or "likes" on trees.
- → Weather Integration: Real-time rainfall or humidity tracking.
- → Gamification/Rewards: Earning digital coins or badges.
- → iOS/Web Versions: Focus is strictly on the Android Kotlin app.
- → Multi-Photo Gallery: Only one "Growth Photo" per status update.

4 Functional Requirements — What the App Must Do

Functional requirements are the things the app MUST DO. If any are missing, the app is incomplete.

FR-01 · Plantation Entry Form Volunteer

Volunteer captures: Species Name, Plantation Type (Seed Ball / Sapling), and Date. The app must validate that a photo is taken and coordinates are fetched before allowing submission.

FR-02 · Image Capture + Image Compression Volunteer

The app uses FusedLocationProviderClient to fetch high-accuracy GPS coordinates at the moment of planting. These coordinates are invisibly linked to the entry to ensure the "Tree Map" is accurate..

FR-03 · Auto Geo-Tagging Volunteer

Volunteer takes a "Day 1" photo. The app automatically compresses the image to ensure fast uploads and low storage impact on the Room DB / Firebase while maintaining enough detail for future growth comparison..

FR-04 · 90-Day Survival Alert System

The app uses WorkManager to schedule a notification exactly 90 days after a planting event. The alert reminds the volunteer to return to the GPS location and "Update Status."

FR-05 · Community Tree Map Volunteer

A Google Maps integration displays all saplings planted by the community. Pins are color-coded: **Green** for "Sprouted/Healthy," **Yellow** for "Needs Care," and **Red** for "Died."

FR-06 · Status Update & Growth Photo Volunteer

Tapping a map pin allows a "Check-up." The volunteer records the current health status and takes a "Growth Photo." The app saves this as a historical timeline for that specific tree.

FR-07 Survival Analytics Dashboard Community

The home screen displays a "Survival Score" for the region (e.g., "75% Survival Rate"). It calculates this by comparing the total number of plants vs. those marked as "Healthy" in the database.

FR-08 · Data-Driven Species Guide Volunteer

A "Best Trees for You" section that ranks species based on local survival data. If "Neem" has a 90% survival rate in the area but "Teak" has 20%, the app recommends Neem to new volunteers.

5 Non-Functional Requirements — How Well the App Must Work

These are not features — they are **quality standards**. They define how fast, safe, and reliable the app must be.

NFR-01 · Performance — Map Fluidity

The "Tree Map" must load and cluster 50+ geo-tagged pins without stuttering. Achieved using Google Maps SDK clustering and efficient Room DB queries to prevent UI thread blocking.

NFR-02 · Compatibility — Rural Device Support

The app must be optimized for Android 8.0 (API level 26) and above to support older smartphones commonly used in rural plantation drives. It must remain functional on low-resolution screen.

NFR-03 · Visual Design — Earthy & Inspiring Theme

Color palette: Forest green, leaf lime, and soil brown. The UI should use rounded corners and organic shapes to feel "natural." Typography should be bold and accessible for outdoor readability.

NFR-04 · Reliability — Persistent Reminders

90-day alerts must trigger even if the phone has been rebooted. This is achieved by setting WorkManager constraints and ensuring the task is persisted in the system's job scheduler.

NFR-05 · Storage — Compressed Images Under 500 KB

Since users may be in remote areas, image compression must keep photos under **300 KB**. This ensures that syncing "Growth Photos" doesn't exhaust the user's mobile data plan.

NFR-06 · Offline Behaviour — Sync-on-Reconnect

In areas with zero signal (forests/fields), the app must allow users to "Plant" offline. Data is saved to Room DB and automatically synced to the cloud once a 4G or Wi-Fi connection is restored.

NFR-07 · Accuracy — GPS Precision

The app must wait for a "High Accuracy" lock (under 10 meters) before allowing a geo-tag. This prevents "drifting" pins and ensures the volunteer can actually find the tree 3 months later

NFR-08 · Battery Optimization — Background Tasks

Location services and background workers must be optimized to prevent battery drain. FusedLocationProvider should be called only when the "New Plant" screen is active, not constantly in the background.

6 Features — Must Have, Good to Have, Out of Scope

Must Have = Without this, the app fails the brief. Build first. **Good to Have** = Makes the app better, not required to pass. **Out of Scope** = Future version only.

Feature	What It Does — Simply	Priority	Week
Plantation Entry Form	Log species, type, and auto-fetch GPS coordinates.	MUST HAVE	Week 1
Geo-Tagging & Camera	Capture "Day 1" photo and link to precise map location.	MUST HAVE	Week 1
Image Compression	Auto-compress photo before upload to save storage and speed.	MUST HAVE	Week 1
Room DB Integration	2-column card grid showing photo, name, price for all products.	MUST HAVE	Week 2
Tree Map (Google Maps)	View all community saplings with color-coded health pins.	MUST HAVE	Week 2
90-Day Status Alarm	WorkManager triggers notification to return and check growth.	MUST HAVE	Week 2
Survival Dashboard	Displays village/region survival % based on user data.	MUST HAVE	Week 2
Earthy Material UI	Green/Brown theme with organic shapes and clear type.	MUST HAVE	Week 3
Splash Screen	Branded entry screen to set the "Sustainability" mood.	GOOD TO HAVE	Week 2
Growth Timeline	View "Day 1" vs "Day 90" photos side-by-side.	GOOD TO HAVE	Week 3
Native Species Guide	List of recommended local trees based on success data.	GOOD TO HAVE	Week 3
Offline Sync	Automatic cloud backup when signal returns.	GOOD TO HAVE	Week 3
Plant Disease AI	Identifies why a tree is dying using the camera.	OUT OF SCOPE	Future
Leaderboard	Ranking of "Top Green Volunteers" in the village.	OUT OF SCOPE	Future

7 Technology Stack

Kotlin	Programming language for Android. Modern, safe, industry-standard.	Free
Android Studio	Official IDE for Android development. Use Electric Eel or later.	Free
Firebase Firestore	Cloud database that stores all product data in real time. No server needed.	Free
Google Maps SDK	Provides the interactive "Tree Map" for geo-tagged saplings.	Free
Glide Library	Efficiently loads and caches "Growth Photos" in the map/list views.	Open Source
Room Database	Local SQLite database to store tree logs and health history offline.	Built-in
Google Stitch	AI-powered UI design tool. Design screens and export to XML layouts.	Free
GitHub	Version control. Push code daily. Required for submission and portfolio.	Free
Material Design 3	Google's design system used to create the "Earthy Green" UI.	Built-in

8 3-Week Delivery Timeline

WEEK 1 · DAYS 1–7

Setup + Logging

Set up Android Studio + Room DB.
Design "Earthy" screens for planting entry.
Implement Camera + GPS tagging logic.
Save initial sapling data to local storage.
✓✓ **Volunteer can log a new plant.**

WEEK 2 · DAYS 8–14

Tracking + Reminders

Build Google Maps integration (Tree Map).
Implement WorkManager for 90-day alerts.
Add health status update forms.
Load map pins from Room/Firestore.
✓✓ **Map shows pins; alerts are scheduled.**

WEEK 3 · DAYS 15–21

Analytics + Polish

Build the Survival Dashboard (Stats).
Polish UI with forest green/brown theme.
Performance test with 50+ map markers.
README + APK generation + GitHub push.
✓✓ **Demo-ready app submitted**

9 Success Criteria — Tick Before Submission

- [ ] Clicking a pin on the Tree Map opens the correct plant details and growth history. — Week 2 · Day 10
- [ ] The app captures and displays the exact Lat/Long within 10 meters of the planting site. — Week 1 · Day 5
- [ ] WorkManager successfully schedules a system notification for future check-ups. — Week 2 · Day 13
- [ ] UI is "Earthy," using greens and browns with high-readability fonts for outdoor use. — Week 3 · Day 16
- [ ] A volunteer can log a tree in a "no-network" zone and have it save to Room DB. — Week 1 · Day 6
- [ ] The dashboard accurately calculates the percentage of "Healthy" vs "Dead" plants. — Week 3 · Day 19
- [ ] Status updates allow for a "Growth Photo" that is saved alongside the original photo. — Week 2 · Day 14
- [ ] The Tree Map remains fluid and responsive when zooming in on high-density clusters — Week 3 · Day 20